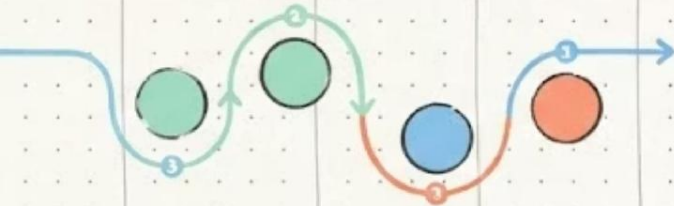


Gestão do Ciclo de Vida da Aplicação

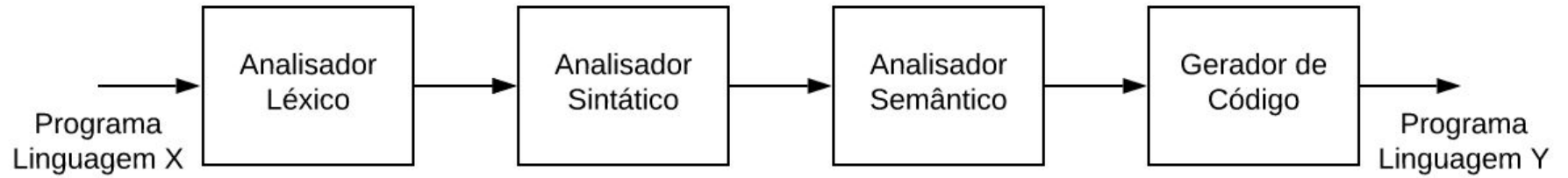
Propriedades de Projeto

Professor Romário da Silva Borges

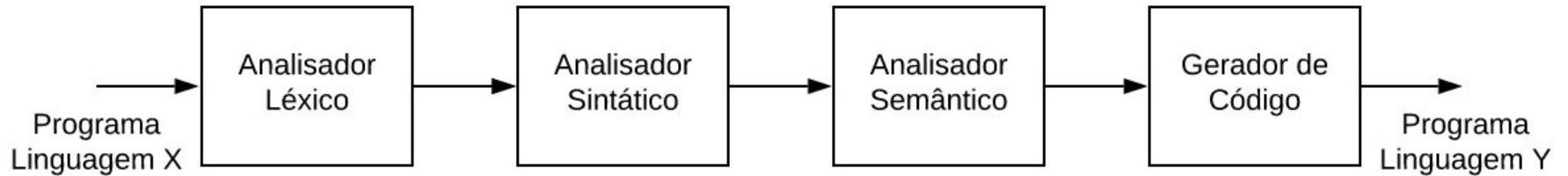
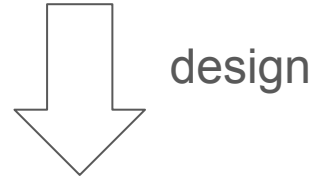
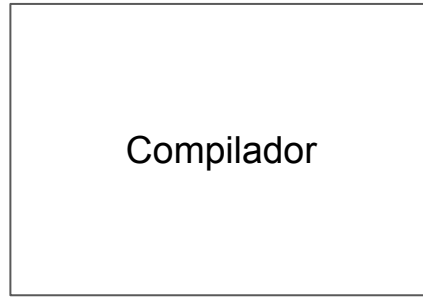


Como implementar um compilador?





O que é design de software?

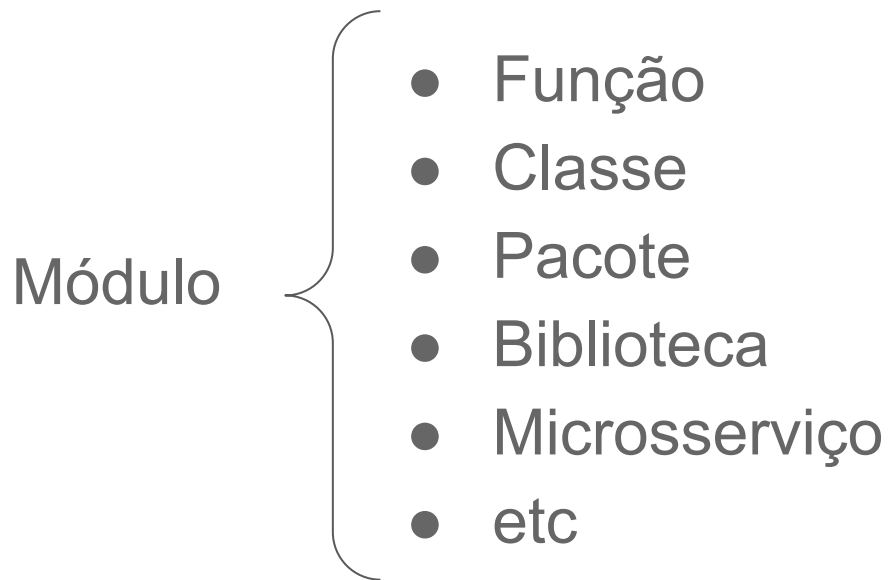


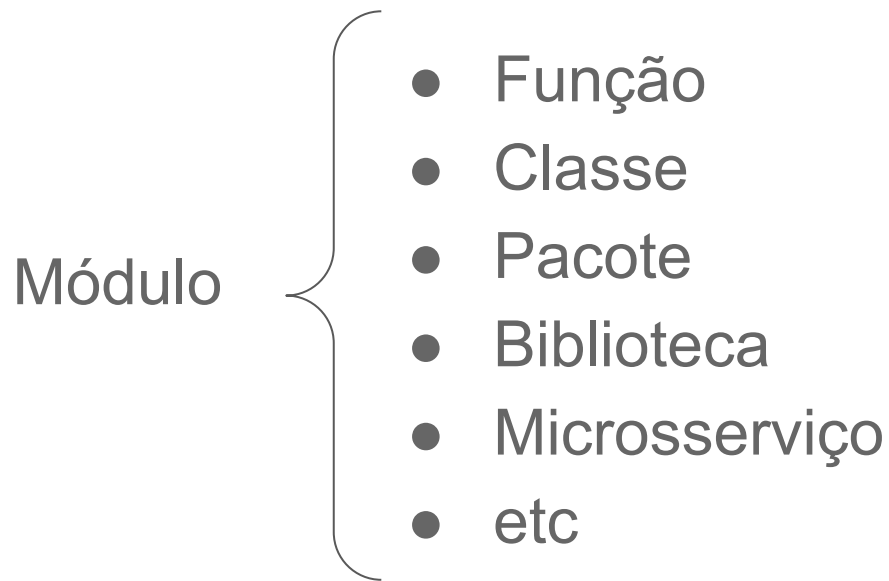
Projeto (Design) de Software

- Dividir para conquistar
- Quebrar um "problema grande" em partes menores

Módulos

- Nome que se dá em design para tais "partes menores"





Módulo = interface + implementação

O que vamos estudar?

Propriedades de "bons projetos" de software

1. Integridade Conceitual
2. Ocultamento de Informação
3. Coesão
4. Acoplamento

Princípios de Projeto

- Responsabilidade Única
- Segregação de Interfaces
- Prefira Interfaces a Classes
- Aberto/Fechado
- Demeter
- Substituição de Liskov

Integridade Conceitual

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of movement and depth. Interspersed within these waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and Java. The code is not the primary focus but adds a technical, digital feel to the overall aesthetic.

Integridade Conceitual em 1 slide



Exemplo



Contra-Exemplo

Por que falta integridade conceitual nesses slides?

Licença CC-BY: permite copiar, distribuir, adaptar etc; porém, **créditos devem ser dados ao autor dos slides**

Engenharia de Software Moderna

Cap. 4 - Modelos

Prof. Marco Tulio de Oliveira Valente

<https://engsoftmoderna.info>, @engsoftmoderna

1

Engenharia de Software Moderna

Cap. 5 - Princípios de Projeto

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>, @engsoftmoderna

2

Licença CC-BY: permite copiar, distribuir, adaptar etc; porém, **créditos devem ser dados ao autor dos slides**

Integridade Conceitual vale para:

- Interface com usuário
- Código
- Projeto
- Implementação
- etc

Exemplo (referente a interface com o usuário)

- Botão "sair" é idêntico em todas as telas
- Se um sistema usa tabelas para apresentar resultados, todas as tabelas têm o mesmo leiaute
- Todos os resultados são mostrados com 2 casas decimais

Tela: Dashboard

Dashboard

Sair

Faturamento (mês)

R\$ 125.430,50

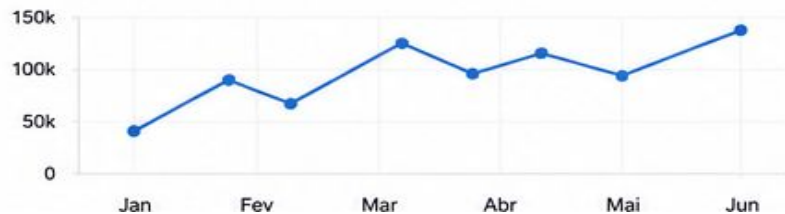
Pedidos

342

Clientes

128

Faturamento últimos 6 meses



Tela: Clientes

Clientes

Sair

Buscar cliente...



+ Novo cliente

Código	Nome	E-mail	Status
001	João Silva	joao@email.com	Ativo
002	Maria Oliveira	maria@email.com	Ativo
003	Pedro Santos	pedro@email.com	Inativo
004	Ana Costa	ana@email.com	Ativo

Tela: Clientes



Clientes

[→ Logout](#)[+ Novo](#)

Código	Nome do Cliente	E-mail	Status
001	João Silva	joao@email.com	Ativo
002	Maria Oliveira	maria@email.com	Ativo
003	Pedro Santos	pedro@email.com	Inativo
004	Ana Costa	ana@email.com	Ativo

[Anterior](#)[1](#)[2](#)[3](#)[Próxima](#)

Tela: Relatórios

Relatórios



Período inicial



Período final

[Aplicar filtro](#)

Resumo do período

Faturamento total

R\$ 125,430.50

Pedidos

342

Ticket médio

R\$ 367.634

Por que "falta" integridade conceitual neste código?

```
discount_rate = 0.10
discountThreshold = 150

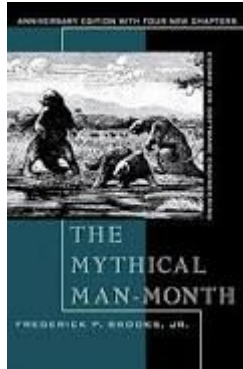
total_orders = len(orders_list)
grossTotal = 0
final_total = 0
largestOrder = 0
high_value_customers = []
```

Exemplos (em nível de projeto/código)

- Todas as páginas usam mesmo framework (mesma versão)
- Se um problema é resolvido com uma estrutura de dados X, todos os problemas parecidos também usam X

Por que integridade conceitual, em nível de código, é importante?

Integridade conceitual é a consideração mais importante no projeto de sistemas -- Fred Brooks



Ocultamento de Informação (Information Hiding)

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of motion and depth. Interspersed within these waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and Java. The code is mostly illegible due to its low opacity and the overall aesthetic of the image.

Origem do conceito (David Parnas, 1972)

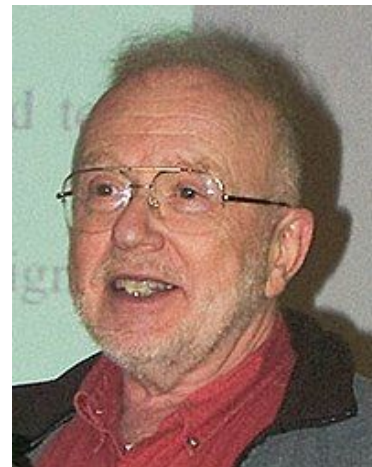
On the Criteria To Be Used in Decomposing Systems into Modules

D.L. Parnas
Carnegie-Mellon University

This paper discusses modularization as a mechanism for improving the flexibility and comprehensibility of a system while allowing the shortening of its development time. The effectiveness of a “modularization” is dependent upon the criteria used in dividing the system into modules. A system design problem is presented and

Introduction

A lucid statement of the philosophy of modular programming can be found in a 1970 textbook on the design of system programs by Gouthier and Pont [1, ¶10.23], which we quote below:¹



Qual o problema com esse programa?

```
decimal[] saldo = { 150, 10, 90 };
```

```
void Depositar(int conta, decimal valor) {  
    saldo[conta] += valor;  
}
```

```
decimal GetSaldo(int conta) {  
    return saldo[conta];  
}
```



Quem usa uma conta deveria precisar saber apenas que pode chamar métodos como `Depositar(...)` e `GetSaldo(...)`. Não deveria precisar saber se, internamente, os saldos estão armazenados em um array, List, Dictionary, banco de dados ou arquivo.

Imagine que você decida trocar: `decimal[] saldos = { 150, 10, 90 };`

Por: `Dictionary<int, decimal> saldos = new(){ { 1, 150 }, { 2, 10 }, { 3, 90 };`

Se várias partes do sistema acessarem, você terá que alterar todo esse código. Com ocultamento de informação, essa mudança fica concentrada apenas na implementação interna da classe ou módulo.

Qual o problema com essa classe?

```
class Conta
{
    public decimal Saldo;

    public Conta(decimal saldo)
    {
        Saldo = saldo;
    }

    public void Depositar(decimal valor)
    {
        Saldo += valor;
    }
}
```

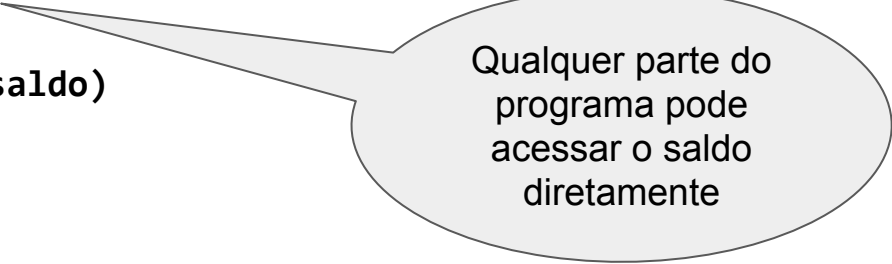
```
Conta conta = new Conta(100);
Console.WriteLine(conta.Saldo);
conta.Depositar(50);
Console.WriteLine(conta.Saldo);
```



```
class Conta
{
    public decimal Saldo;

    public Conta(decimal saldo)
    {
        Saldo = saldo;
    }

    public void Depositar(decimal valor)
    {
        Saldo += valor;
    }
}
```



Qualquer parte do
programa pode
acessar o saldo
diretamente

```
Conta conta = new Conta(100);
Console.WriteLine(conta.Saldo);
conta.Depositar(50);
Console.WriteLine(conta.Saldo);
```

```
class Conta
```

```
{
```

```
    private decimal saldo;
```

```
    public Conta(decimal saldo) { Saldo = saldo; }
```

```
    public void Depositar(decimal valor){ Saldo += valor;}
```

```
    public decimal GetSaldo() { return saldo; }
```

```
}
```

```
Conta conta = new Conta(100);
```

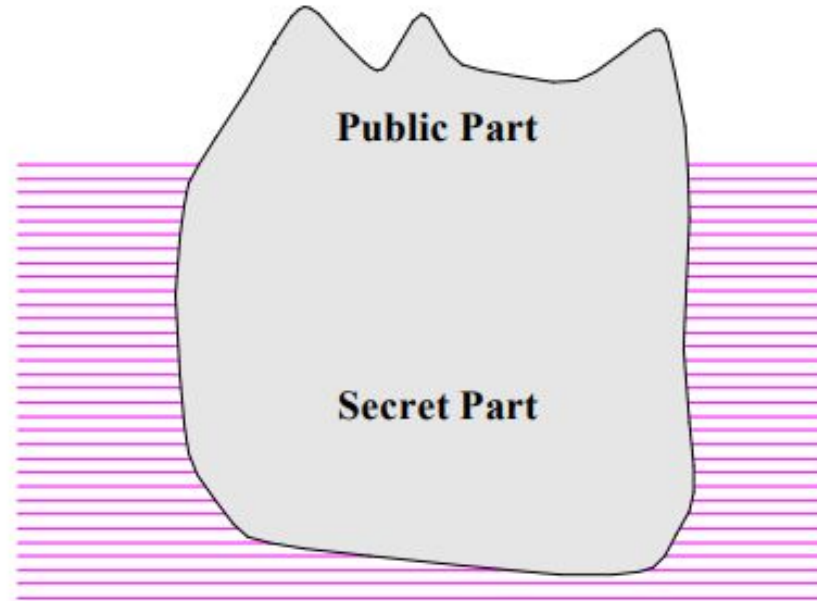
```
Console.WriteLine(conta.GetSaldo());
```

```
conta.Depositar(50);
```

```
Console.WriteLine(conta.GetSaldo());
```

Acesso privado
agora

Information Hiding em 1 slide



Fonte: Bertrand Meyer, Object-oriented software construction, 1997 (pág. 51)

Ocultamento de Informação

- Classes devem ocultar detalhes internos de sua implementação (usando modificador **private**)
- Principalmente aqueles sujeitos a mudanças

Qual o problema com essa classe?

```
public class Estacionamento {  
  
    public Hashtable<String, String> veiculos;  
  
    public Estacionamento() {  
        veiculos = new Hashtable<String, String>();  
    }  
  
    public static void main(String[] args) {  
        Estacionamento e = new Estacionamento();  
        e.veiculos.put("TCP-7030", "Uno");  
        e.veiculos.put("BNF-4501", "Gol");  
        e.veiculos.put("JKL-3481", "Corsa");  
    }  
}
```



```
public class Estacionamento {  
    private Hashtable<String, String> veiculos = new Hashtable<>();  
  
    public Estacionamento() {  
        veiculos = new Hashtable<String, String>();  
    }  
    public void adicionarVeiculo(String placa, String modelo) { veiculos.put(placa,  
modelo); }  
  
    public String buscarVeiculo(String placa) { return veiculos.get(placa); }  
  
    public static void main(String[] args) {  
        Estacionamento e = new Estacionamento();  
  
        e.adicionarVeiculo("TCP-7030", "Uno");  
        e.adicionarVeiculo("BNF-4501", "Gol");  
        e.adicionarVeiculo("JKL-3481", "Corsa"); System.out.println(e.buscarVeiculo("TCP-7030"));  
    }  
}
```

o cliente não sabe
que internamente
existe Hashtable

Comparação com um sistema manual de estacionamento

A photograph of a hand holding a spiral-bound notebook. The notebook is open to a page titled "PARKING PERMIT LOG". The form has columns for "Parking Permit Number", "Lot and/or Space Designation (if applicable)", "ISSUED TO (Name of Owner or Driver)", "Age, Sex, or Department", and "Date Issued". The first row is filled out with the following information: "990-8571", "A571", "Dan Johnson", "Warehouse", and "5/17/22".

Parking Permit Number	Lot and/or Space Designation (if applicable)	ISSUED TO (Name of Owner or Driver)	Age, Sex, or Department	Date Issued
990-8571	A571	Dan Johnson	Warehouse	5/17/22

Como melhorar o design da classe
Estacionamento?

Sistema manual: o cliente solicita entrada, consulta ou saída ao atendente; não manipula os registros diretamente.

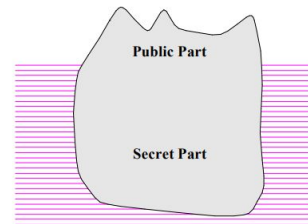
Bom design: a classe Estacionamento deve fazer o mesmo, ocultando sua estrutura interna e expondo apenas operações como entrar(), sair() e consultar().

Qual o problema com o método getBookISBN?

```
import java.sql.ResultSet;

public class LibraryServices {

    public ResultSet getBookByISBN(String isbn) {
        ...
    }
}
```



```
import java.sql.ResultSet;

public class LibraryServices {

    public Book getBookByISBN(String isbn) {
        ...
    }

}
```

Código cliente:

```
Book livro = service.getBookByISBN("123");

System.out.println(livro.getTitulo());
```

Problema: o método expõe ResultSet, revelando detalhes internos de persistência.

Solução: retornar um objeto de domínio, como Book, ocultando a tecnologia usada para acessar os dados.

Significados do termo interface

The background of the slide features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement and depth. Interspersed within these waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and C++, which are slightly blurred to maintain focus on the title.

Significados do termo interface

- Interface: parte visível de um módulo (nosso uso!)
- Interface: construção de uma linguagem (palavra reservada)
- Existe ainda um terceiro significado:
 - Interface com o usuário (UI)
 - Interface com o usuário gráfica (GUI)
 - Interface mobile
 - etc

Interface em Java

```
interface Chat {  
    void sendMsg(String);  
}  
  
class WhatsApp implements Chat {  
    public void sendMsg(String m) {  
        ...  
    }  
}  
  
class Telegram implements Chat {  
    public void sendMsg(String m) {  
        ...  
    }  
}
```

Código mantido por um time X

Interface em Java

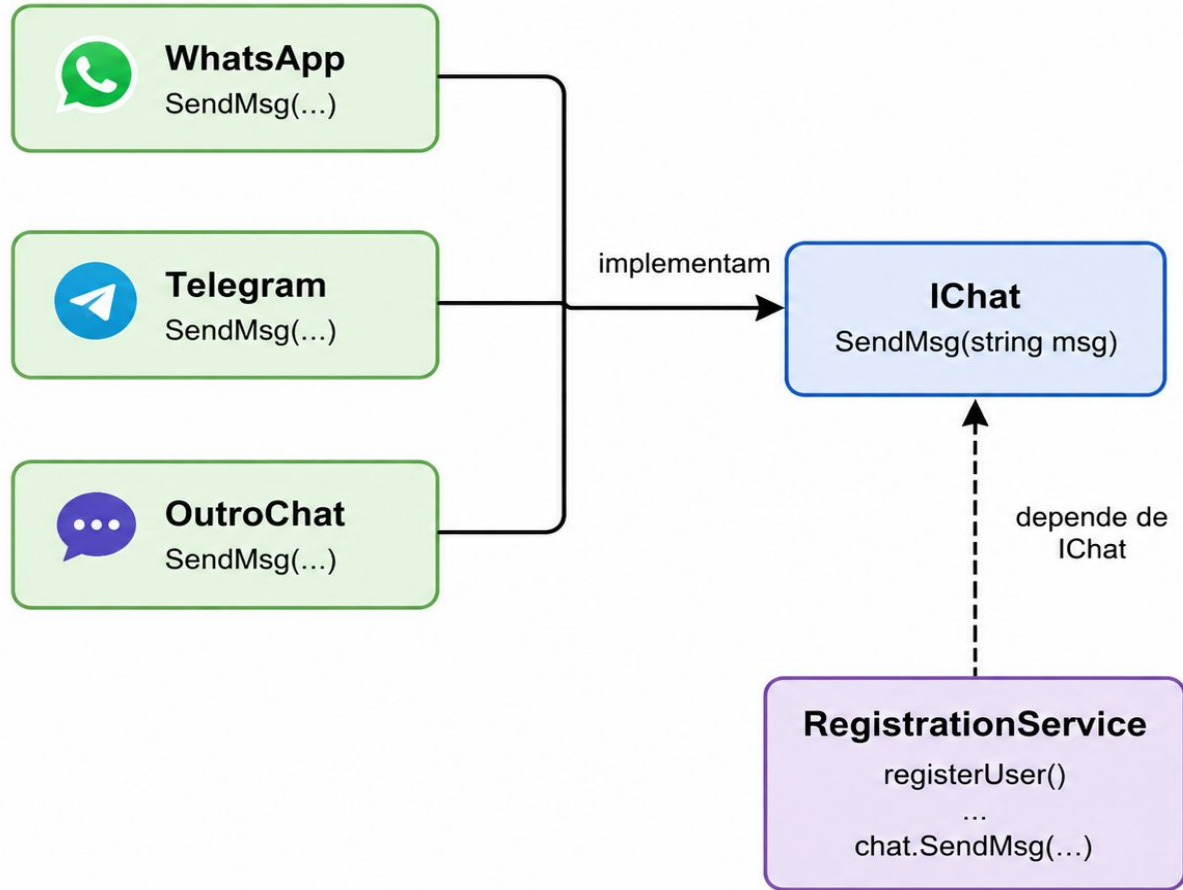
```
interface Chat {  
    void sendMsg(String);  
}  
  
class WhatsApp implements Chat {  
    public void sendMsg(String m) {  
        ...  
    }  
}  
  
class Telegram implements Chat {  
    public void sendMsg(String m) {  
        ...  
    }  
}
```

Código mantido por um time X

WhatsApp Telegram etc

```
class RegistrationService {  
  
    private Chat chat;  
  
    public RegistrationService(Chat chat) {  
        this.chat = chat;  
    }  
  
    public void registerUser() {  
        ...  
        chat.sendMsg(...);  
        ...  
    }  
}
```

Código mantido por um time Y



Interface em C#

```
public interface IChat {  
    void SendMsg(string msg);  
}  
  
public class WhatsApp : IChat {  
    public void SendMsg(string msg) { ... }  
}  
  
public class Telegram : IChat {  
    public void SendMsg(string msg) { ... }  
}
```

Código mantido por um time X

Interface em C#

```
public interface IChat {  
    void SendMsg(string msg);  
}  
  
public class WhatsApp : IChat {  
    public void SendMsg(string msg) {  
        ...  
    }  
}  
  
public class Telegram : IChat {  
    public void SendMsg(string msg) {  
        ...  
    }  
}
```

Código mantido por um time X

WhatsApp Telegram etc

```
public class RegistrationService {  
    private IChat chat;  
  
    public RegistrationService(IChat chat)  
    {  
        this.chat = chat;  
    }  
  
    public void RegisterUser() {  
        ...  
        chat.SendMsg(...);  
    }  
}
```

Código mantido por um time Y

Quem configura a aplicação é que decide qual implementação entregar:

```
IChat chat = new WhatsApp();  
RegistrationService service = new RegistrationService(chat);
```

Ou:

```
IChat chat = new Telegram();  
RegistrationService service = new RegistrationService(chat);
```

Para o time Y, nada muda:

```
service.RegisterUser();
```

Sem o uso de Interface, teríamos algo assim:

```
public class RegistrationService {  
    private WhatsApp whatsapp;  
    private Instagram instagram;  
  
    public RegistrationService() {  
        whatsapp = new WhatsApp();  
        instagram = new Instagram();  
    }  
  
    public void RegisterUser() {  
        ...  
        whatsapp.SendMsg("Usuário cadastrado");  
        instagram.SendMsg("Usuário cadastrado");  
    }  
}
```

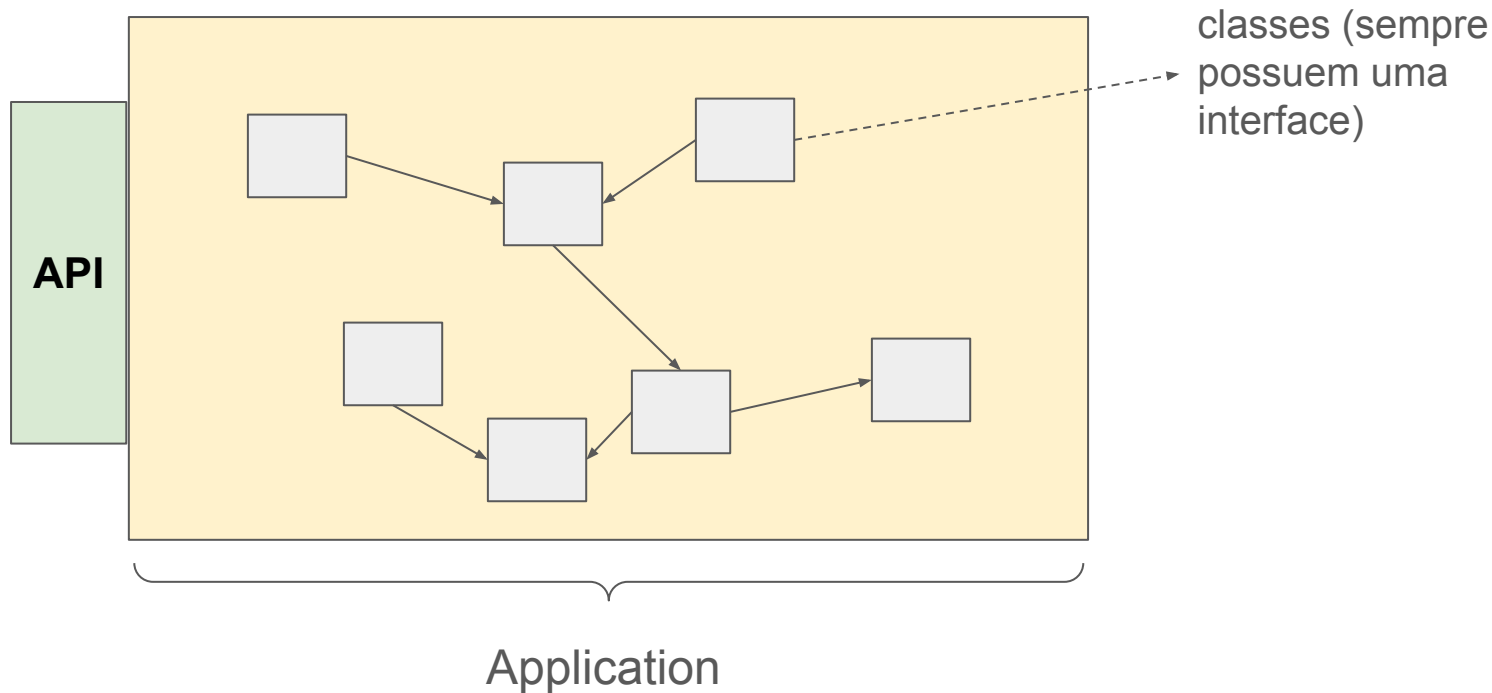
- Existe dependência direta;
- Precisa conhecer as implementações concretas;
- Alto acoplamento



Mesmo que uma classe não implemente uma interface (palavra reservada), ela possui uma interface (seus métodos públicos)

Qual a diferença entre API e interface?

Exemplo: GPT4.5 da openAI



Outro nome: encapsulamento

- Alguns autores, preferem o nome encapsulamento, mas com o mesmo significado de ocultamento de informação

Encapsulation

See information hiding.

Glossário do livro Object-Oriented Software Construction.
Bertrand Meyer (p. 1195)

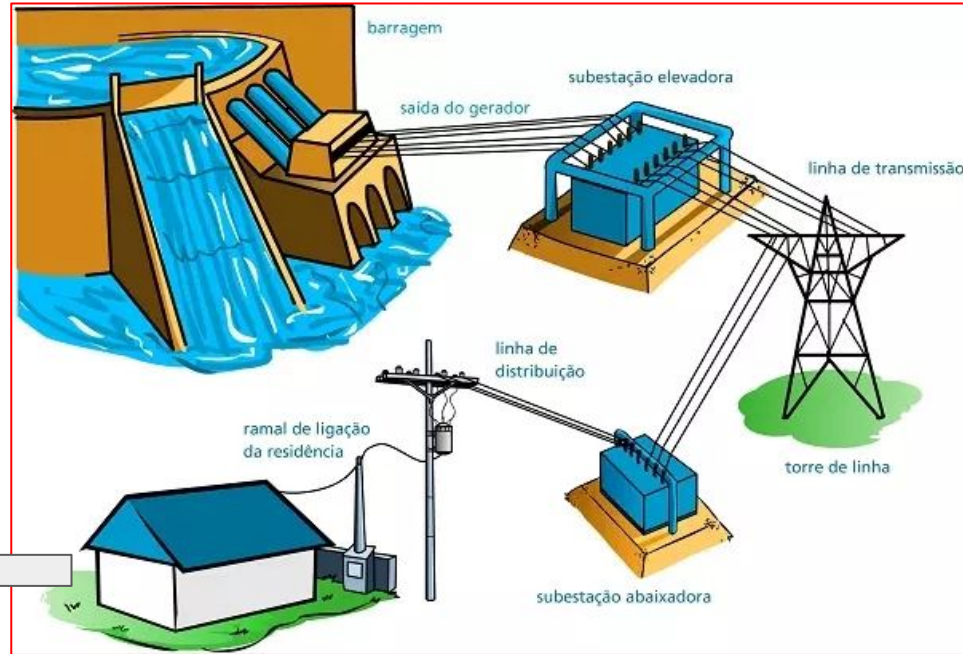
Information Hiding em 1 slide

Implementação

Interface



110 volts



“Programe para uma interface (abstração), e não para uma implementação.”

Coesão

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of movement and depth. Scattered throughout the background are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and Java. The code is mostly illegible due to its low opacity and the overall aesthetic of the image.

Coesão

- Uma classe deve ter uma única função
- Oferecer um único serviço
- Vale também funções, métodos, pacotes, etc.

Contra-exemplo 1

```
float sin_or_cos(double x, int op) {  
    if (op == 1)  
        "calcula e retorna seno de x"  
    else  
        "calcula e retorna cosseno de x"  
}
```



Correção

```
float seno(double x) {  
    return sin(x);  
}  
  
float cosseno(double x) {  
    return cos(x);  
}
```



Contra-exemplo 2

```
class Estacionamento {  
    ...  
    private String nome_gerente;  
    private String fone_gerente;  
    private String cpf_gerente;  
    private String endereco_gerente;  
    ...  
}
```



Correção

```
class Estacionamento {  
    private Gerente gerente;  
}  
  
class Gerente {  
    private String nome;  
    private String fone;  
    private String cpf;  
    private String endereco;  
}
```



Exemplo

```
class Stack<T> {  
    boolean empty() { ... }  
    T pop() { ... }  
    push (T) { ... }  
    int size() { ... }  
}
```



Todos esses métodos manipulam os elementos da Pilha

Acoplamento

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of motion and depth. Scattered across the waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and Java. The code is mostly illegible due to its low opacity and the motion blur of the waves, but it adds a technical, digital feel to the overall composition.

Acoplamento

- Nenhuma classe é uma ilha ...
- Classes dependem umas das outras (chamam métodos de outras classes, estendem outras classes, etc)
- A questão principal é a **qualidade** desse acoplamento
- Dois tipos:
 - Acoplamento aceitável ("bom")
 - Acoplamento ruim

Acoplamento Aceitável

- Classe A usa uma classe B e:
 - B oferece um serviço útil para A
 - B possui uma interface estável
 - A somente chama métodos da interface de B

```
import java.util.Hashtable;

public class Estacionamento {

    private Hashtable<String,String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

    public void estaciona(String placa, String veiculo) {
        veiculos.put(placa, veiculo);
    }

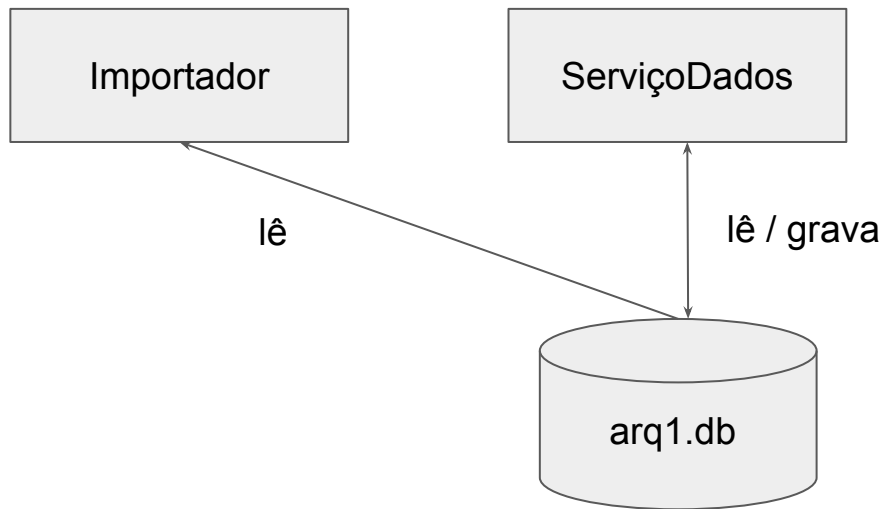
    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.estaciona("TCP-7030", "Uno");
        e.estaciona("BNF-4501", "Gol");
        e.estaciona("JKL-3481", "Corsa");
    }
}
```



Estacionamento depende (está acoplada) a Hashtable, mas esse acoplamento é aceitável

Acoplamento Evolutivo (Ruim)

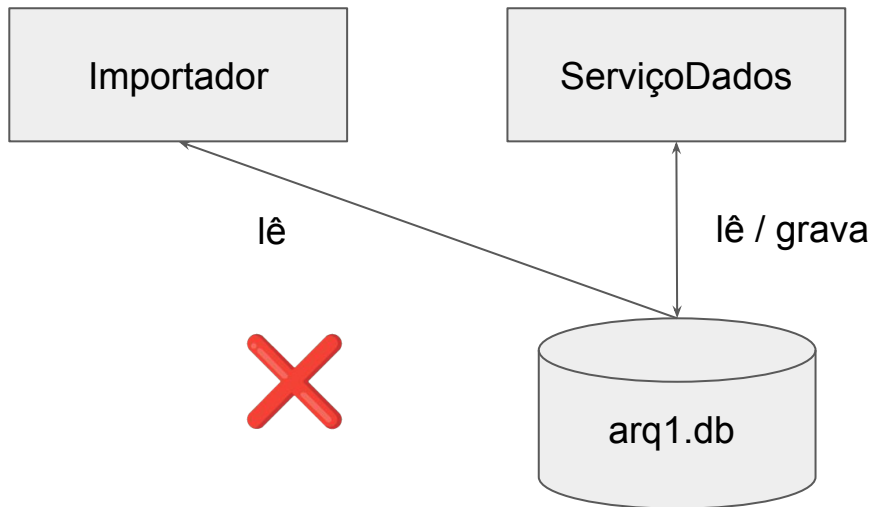
Classe Importador está acoplada em ServiçoDados?



Acoplamento Ruim (ou Evolutivo)

- Classe Importador não referencia diretamente ServiçoDados
- Mas mudanças em ServiçoDados podem impactar a classe Importador

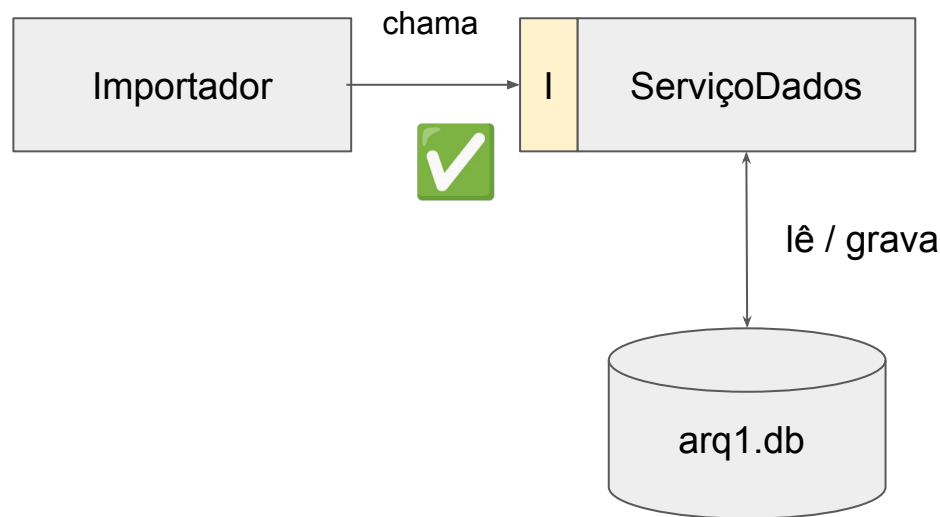
Como transformar um acoplamento ruim (evolutivo) em um acoplamento bom?



```
class Importador {  
    // espera um arquivo CSV:  
    // Nome;CPF;Email  
}  
  
class ServicoDados {  
    // lê/grava o arquivo CSV:  
    // Nome;CPF;Email  
}
```

Elas não se chamam diretamente, mas se o ServicoDados mudar o formato para *NomeCompleto Documento e Email*, o Importador pode quebrar.

Como transformar um acoplamento ruim (evolutivo) em um acoplamento bom?



Alguém apagou 11 linhas de código Open Source e quebrou a internet; o caso trouxe várias lições que ninguém quis aprender

Do caso 'left-pad' ao Log4Shell fica claro que a internet funciona graças a um grupo pequeno de geeks que fazem tudo na base do amor à arte

30 mai 2025 - 17h14 (atualizado em 1/6/2025 às 12h23)

Compartilhar 

[Exibir comentários](#)

Por: Sofia Bedeschi / Licenciado de Xataka

Recomendação comum em projeto de software:

Maximize a coesão, minimize o acoplamento

Mas cuidado: minimize (ou elimine) principalmente o acoplamento ruim

Atividades

The background features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement and depth. Faint, semi-transparent snippets of code are visible, particularly on the left and right sides, suggesting a technical or programming theme. The overall aesthetic is modern and digital.

1. Suponha que você ficou responsável pela implementação de um sistema que vai demandar ~100 KLOC.

Proponha hipoteticamente um design para sua implementação que:

- tenha a pior coesão possível
- mas, ao mesmo tempo, tenha o melhor acoplamento possível.

2. (a) Você consegue dizer qual valor o seguinte programa vai imprimir (sem conhecer o código de f)?

```
var g = 1; // global  
f();  
print g;
```



(b) Usando esse programa como exemplo, explique porque variáveis globais dão origem a designs ruins?

(c) Sistemas com muitas variáveis globais não atendem a qual propriedade de projeto?

3. Sejam duas classes A e B que:

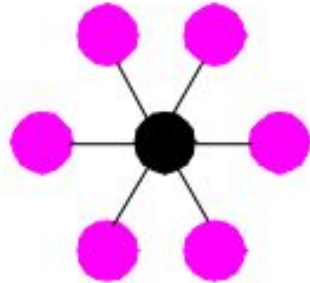
- estão implementadas em diretórios diferentes
- a classe A possui uma referência no seu código para B.

Então, sempre que um programador precisa, como parte de uma tarefa de manutenção, modificar classes A e B que atendem a tais critérios ele conclui a tarefa movendo B para o mesmo diretório de A.

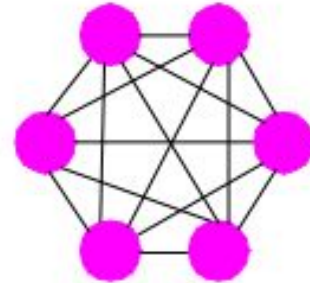
(a) Agindo dessa maneira, o programador estará melhorando qual propriedade de projeto (quando medida entre diretórios)?

(b) E qual propriedade é afetada de modo negativo?

4. Genericamente falando, qual dos seguintes projetos é melhor? Justifique (nodos = classes; arestas = dependências)



(A)



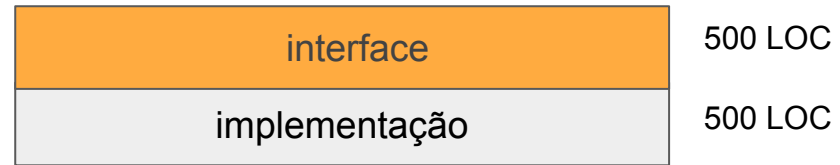
(B)

Fonte: Bertrand Meyer, Object-oriented software construction, 1997 (pág. 47)

5. Qual dos seguintes módulos é melhor? Justifique.



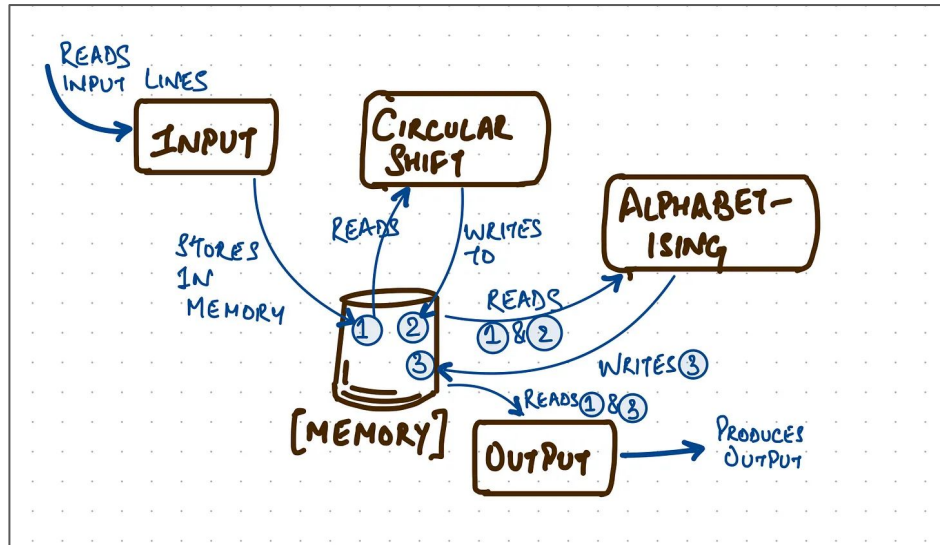
(A)



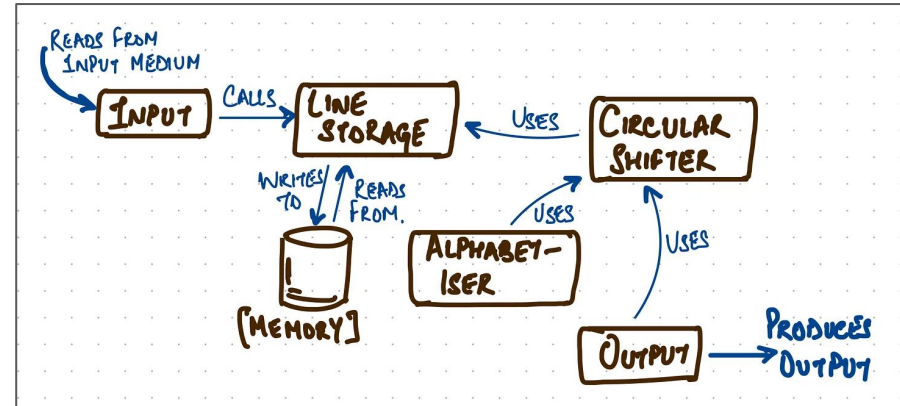
(B)

Inspirado em conceitos propostos no livro A Philosophy of Software Design. John Ousterhout

6. Seguem duas modularizações de um programa que lê um conjunto de linhas da entrada, cria todos os “shift circulares” dessas linhas e depois imprime os shifts em ordem alfabética. Qual das duas modularizações é melhor? Qual propriedade de projeto ela atende (em detrimento da outra)?



Modularização I



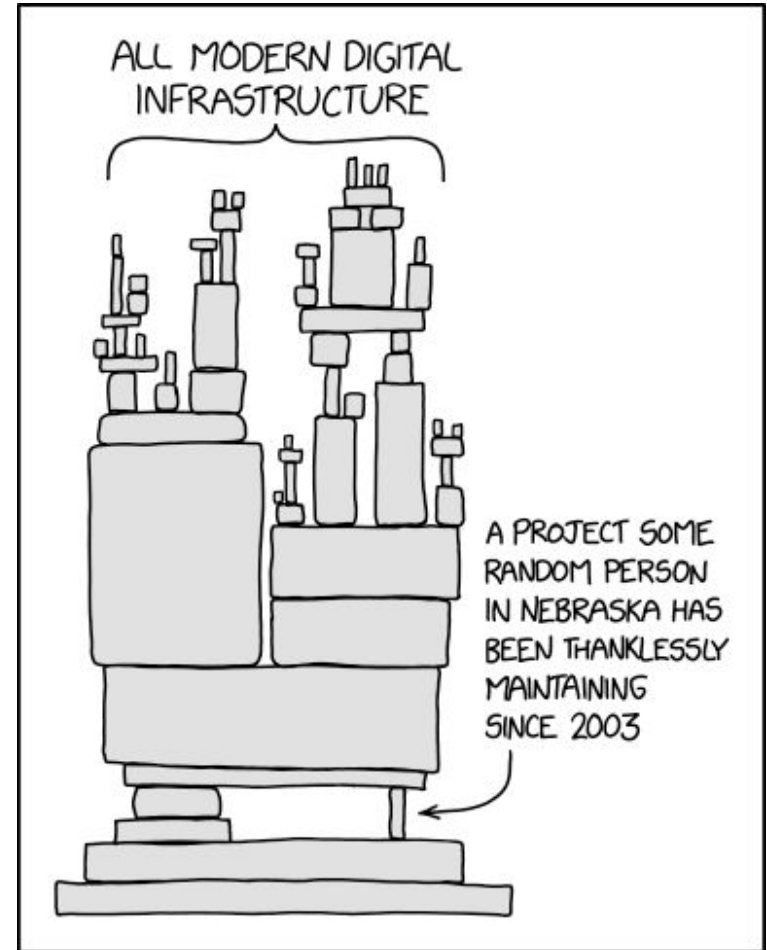
Modularização II

7. Suponha dois métodos *f* e *g*. Existe um **acoplamento temporal** entre eles quando para chamar *g* temos que chamar antes *f*.

- (a) Dê um exemplo aceitável e bastante comum de acoplamento temporal (ou seja, dê nomes concretos de métodos *f* e *g*).
- (b) O acoplamento temporal do seguinte código é aceitável ou ruim? Se for ruim, sugira uma refatoração para melhorar o código.

```
var circle = new Circle();  
circle.setRadius(5);  
circle.getArea();
```

8. A seguinte figura, de certo modo, ilustra os problemas derivados de qual propriedade de projeto?



Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: <https://engsoftmoderna.info>.

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

